Практическая работа № 3. Оптимизация изображений

Цель работы: изучить способы уменьшения размера изображений и проверить, как оптимизация влияет на загрузку веб-страниц.

Задание 1. Сжатие изображения

Шаг 1. Для работы была выбрана фотография вечернего Чикаго. Исходный файл называется 132.jpg. По данным загрузки на компьютере его размер составлял 20,9 МБ.



Рисунок 1 — Исходная фотография Чикаго

Шаг 2. Для оптимизации уменьшил разрешение изображения и сохранил его в JPEG с более сильным сжатием. Получился файл foto chikago.jpg размером 1280 × 854 пикселя.

Шаг 3. Сравнил объём двух файлов. После оптимизации файл foto chikago.jpg занимает около 198 КБ. Размеры обоих файлов показаны на скриншоте.



Рисунок 2 — Оптимизированная фотография

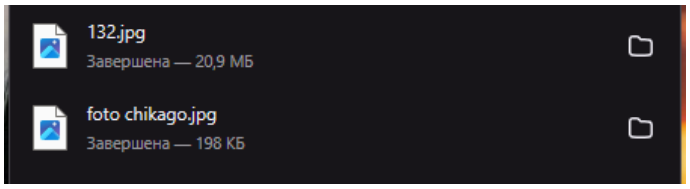


Рисунок 3 — Размеры файлов до и после оптимизации

Шаг 4. Результаты сравнения приведены в таблице 1.

Таблица 1 — Сравнение изображений

Параметр	**Исходное**	**Оптимизированное**
Имя файла	132.jpg	foto chikago.jpg
Формат	JPEG	JPEG
Размеры, пикс.	2048 × 1367	1280 × 854
Размер файла	20,9 МБ	198 КБ
Источник размера	Скриншот загрузки	Скриншот загрузки

Шаг 5. По данным на скриншоте файл стал легче примерно в 106 раз, экономия составила около 99 %. На небольшом экране разницу в качестве заметить сложно. При сильном увеличении у сжатого файла может быть меньше мелких деталей.

Самостоятельная работа. Оценка загрузки

Для проверки создал на GitHub Pages две страницы: original.html с исходной фотографией и optimized.html со сжатой. Обе страницы содержат

заголовок и одно изображение. В Firefox открыл инструменты разработчика, вкладку «Сеть», отключил кэш и обновил страницы.

На рисунке 4 показана загрузка original.html. Статус запроса изображения — 200. Было передано 21,87 МБ, а полная загрузка страницы заняла 3,03 секунды.

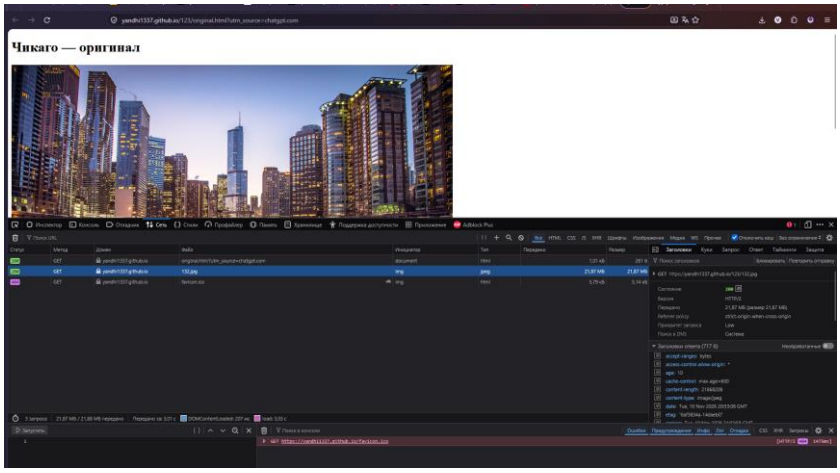


Рисунок 4 — Загрузка исходного изображения в Firefox

На рисунке 5 показана проверка optimized.html. Передано 203,24 КБ, общая загрузка страницы — 0,64 секунды. В обоих тестах фотография отобразилась.

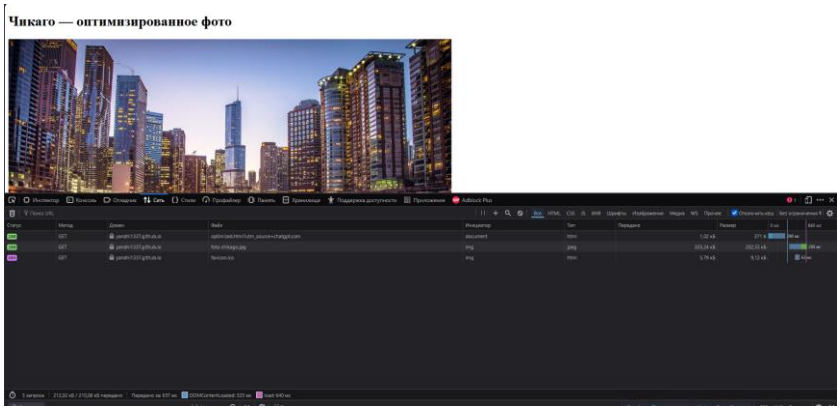


Рисунок 5 — Загрузка оптимизированного изображения в Firefox

Таблица 2 — Сравнение загрузки страниц в Firefox

Показатель	**Оригинал**	**После сжатия**
Передано для изображения	21,87 МБ	203,24 КБ
Полная загрузка	3,03 сек.	0,64 сек.

страницы		
DOMContentLoaded	207 мс	325 мс
Статус изображения	200	200

В этом замере время полной загрузки уменьшилось на 2,39 секунды: 3,03 / 0,64 ≈ 4,7. Получается, что страница со сжатой фотографией загрузилась примерно в 4,7 раза быстрее. При повторных запусках результат может немного отличаться.

Ответы на вопросы к заданию 1

1. Для чего нужна оптимизация изображений? Чтобы страницы быстрее загружались и расходовали меньше интернет-трафика.

2. Какой тип сжатия подходит для фотографии? Обычно с потерями, потому что он заметно уменьшает размер JPEG и WebP.

3. Какие инструменты можно использовать? TinyPNG, Squoosh, GIMP, ImageMagick и другие редакторы.

4. Какие дефекты бывают после сжатия? Размытие, квадраты на однородных участках и потеря деталей. Помогает умеренное качество.

5. Какие метрики подходят для проверки? Размер файла, время загрузки, LCP и показатели визуального сходства SSIM или PSNR.

6. Влияет ли содержание изображения на выбор формата? Да. Для фотографий подходит JPEG или WebP, для схем и прозрачности — PNG, SVG или WebP.

7. Какой показатель связан с отрисовкой крупного изображения? LCP — время появления самого крупного элемента в видимой части страницы.

8. Что такое lazy loading? Отложенная загрузка изображений, которые пока находятся за пределами экрана.

9. Можно ли оптимизировать JPEG и PNG? Да, но способы разные: у JPEG меняют качество, у PNG — палитру и степень сжатия.

10. Что делать, если изображение всё ещё слишком большое? Уменьшить размеры, изменить качество или попробовать WebP/AVIF.

Задание 2. Адаптивное отображение изображения

Для второго задания подготовил простой макет страницы о Чикаго. На странице есть шапка, фотография, текст и три информационные карточки. Макет меняется в зависимости от ширины экрана.

На компьютере ссылки меню располагаются в одну строку, а карточки — в три колонки. На экране телефона навигация скрывается, карточки становятся друг под другом, изображение занимает доступную ширину.

При вёрстке использованы тег viewport, CSS Grid и медиазапрос @media. Изображению задан атрибут alt, а его ширина в CSS ограничена контейнером.

Листинг 1 — Основные правила адаптивной вёрстки

```
<meta name="viewport" content="width=device-width, initial-scale=1">
<img src="foto chikago.jpg" alt="Вечерний Чикаго"
    width="1280" height="854">
<style>
 img { max-width: 100%; height: auto; }
 .cards { display: grid; grid-template-columns: repeat(3, 1fr); }
 @media (max-width: 700px) {
  .cards { grid-template-columns: 1fr; }
  nav { display: none; }
 }
</style>
```

В папке с практической работой также подготовлен файл chicago_demo.html. Его можно открыть в браузере и проверить на разных размерах окна.

Ответы на вопросы к заданию 2

1. Чем адаптивный дизайн отличается от отзывчивого? Адаптивный использует заранее заданные варианты компоновки, а отзывчивый плавно подстраивается под ширину экрана.

2. Как работают медиазапросы? Браузер применяет CSS-правила, когда ширина экрана соответствует условию @media.

3. Что означает подход Mobile First? Сначала создают стили для телефона, затем дополняют их для широких экранов.

4. Что происходит с боковыми блоками на телефоне? Они могут переноситься ниже основного содержимого или скрываться.

5. Как изменить расположение меню? Использовать flex, grid и медиазапросы; на телефоне можно сделать компактное меню.

6. Скрытый через CSS элемент перестаёт загружаться? Не обязательно. Файлы могут загружаться даже при display: none.

7. Для чего нужны srcset, sizes и picture? Они помогают браузеру выбрать картинку нужного размера или формата.

8. Почему SVG удобен для иконок? Его можно масштабировать без потери чёткости.

9. Для чего нужен viewport? Он сообщает браузеру ширину области просмотра и помогает правильно отображать мобильную версию.

10. Как проверить мобильную версию? Открыть инструменты разработчика браузера и включить режим устройств, затем по возможности проверить на телефоне.

Вывод

В ходе работы сравнил исходную и сжатую фотографии Чикаго. По данным загрузки размер файлов изменился с 20,9 МБ до 198 КБ. Для проверки разместил две версии страницы на GitHub Pages. В Firefox при отключённом кэше полная загрузка сократилась с 3,03 до 0,64 секунды, то есть примерно в 4,7 раза. Также разобрал основные правила адаптивной вёрстки. Оптимизация заметно уменьшила трафик и ускорила загрузку страницы в проведённом тесте.